
Graph-based Food Recommendation System

Group 6

Sun Jiayi
122090484@link.cuhk.edu.cn

Pan Le
122090412@link.cuhk.edu.cn

Qian Jingshuai
122090428@link.cuhk.edu.cn

Zhong Peirong
122090809@link.cuhk.edu.cn

1 Introduction

With the widespread availability of personalized data and the rise of various food-related applications, it has become possible to develop systems capable of recommending food based on user preferences. However, traditional food recommendation systems often rely solely on basic user preference information, unable to fully capture the complexity of users' taste preferences. To address this limitation, this project aims to apply graph-based algorithms into food recommendation in order to model intricate relationships between user features, food features, and user-food ratings. Section 2 shows how the project collect and pre-process data. Section 3 demonstrates the applied graph-based algorithms and a collaborative filtering method as a baseline. Section 4 presents the algorithms' performance on pre-processed data with RMSE and MAE. Section 5 compares and analyzes the algorithms by mainly pointing out their advatanges and limitations. Section 6 concludes this project.

2 Data Preparation

This project collected data from 418 questionnaire responses. The questionnaire assessed: (1) Flavor preferences (spicy, sweet, sour), (2) Food category preferences (vegetables vs. meat), (3) Acceptance of distinctive food flavors (raw, cold, pungent), (4) Acceptance of diverse culinary traditions, and (5) Item-specific ratings (1=strongly dislike to 5=strongly like), with 0 indicating unevaluated items.

Raw data from questionnaire responses were processed in following ways:

1. Each food's features were encoded into a binary vector in $\{0, 1\}^{16}$. For each feature, 1 indicates its presence while 0 indicates its absence.
2. Each user's features were encoded into a binary vector in $\{0, 1\}^{22}$. The project one-hot encodes users' genders denoting female as 0 while male as 1. Also, the project divided users' ages into 4 groups, $(0, 18]$, $(18, 30]$, $(30, 50]$ and $(50, \infty)$ and one-hot encoded them. As for other features, 1 indicates user acceptance or preference, while 0 represents the opposite.

These processed datasets will be used to construct the user and food graph structure, which serves as the foundation for the graph-based recommendation algorithm.

3 Methods: Graph-based Algorithms

3.1 GAT-based User Graph Algorithm

We construct an undirected graph based on questionnaire features between users (such as dietary preferences, lifestyle, etc.) using the **K-Nearest Neighbors (KNN)** algorithm. Each node represents a user or a food, and edges are formed based on the top K similarity of feature vectors.

Then for each user node u_i in the graph, we calculate attention weights using its neighbor nodes (e.g., u_j, u_k):

$$\alpha_{ij}^{(h)} = \text{softmax}_j \left(\text{LeakyReLU} \left(\mathbf{a}_h^\top \left[W_h^{(1)} x_i \parallel W_h^{(1)} x_j \right] \right) \right) \quad (1)$$

where $W_h^{(1)}$ and $\mathbf{a}_h$ are learnable parameters, and $\parallel$ denotes the concatenation operation. This step measures the importance of neighbor nodes in updating the representation of the target node.

Each node aggregates the features of its neighbors via multiple attention heads to form its final representation:

$$z_i = \parallel_{h=1}^H \sum_{j \in N(i)} \alpha_{ij}^{(h)} W_h^{(1)} x_j \quad (2)$$

A linear layer is then applied to transform the representation into h_i , which is used for subsequent rating prediction:

$$\hat{R}_i = W_{\text{out}} h_i \quad (3)$$

The model is trained using mean squared error (MSE) loss, computed only at positions specified by `train_mask`:

$$\mathcal{L} = \frac{1}{|\text{train_mask}|} \sum_{i \in \text{train_mask}} \left\| \hat{R}_i - R_i \right\|^2 \quad (4)$$

3.2 Two-stage Architecture Algorithm

In this algorithm, graphs are constructed separately for users and foods using the KNN algorithm for each node (user or food). First we compute the Euclidean distance between any two nodes and select the nearest $k + 1$ neighbors (including itself) for each node. Edges in the graph indicate similarity in features between two users/foods. Then we construct a symmetric adjacency matrix A_{sym} , and normalize it to obtain:

$$A_{\text{norm}} = D^{-\frac{1}{2}} (A_{\text{sym}} + I) D^{-\frac{1}{2}} \quad (5)$$

The final result consists of two graphs: a user graph (nodes: users; edges: similar users) and a food graph (nodes: foods; edges: similar foods).

We use a multi-head attention mechanism, where each node (user or food) first undergoes a linear transformation. Attention weights are then computed between each node and its neighbors. Each attention head produces a d' -dimensional output vector, and the outputs from all K heads are concatenated to form a final node embedding of dimension $K \cdot d'$ (Veličković et al., 2017).

This process results in embedding vectors for all users and food items, denoted as emb_{user} and emb_{food} , respectively. For each user i and food j , we construct a combined representation by concatenating their embeddings:

$$z_{ij} = [\text{emb}_{\text{user}_i} \parallel \text{emb}_{\text{food}_j}] \quad (6)$$

which yields a vector of dimension $2K \cdot d'$.

The concatenated vector z_{ij} is then passed into a multi-layer perceptron (MLP). The MLP consists of 2 hidden layers with ReLU activations and outputs a continuous score representing the predicted rating. To ensure the score falls within a valid rating scale, we apply `clamp` to restrict the value to $[1, 5]$, and then use `round` to convert it to the nearest integer as the final predicted rating.

3.3 User-Food Heterogeneous GCN Algorithm

This model constructs a heterogeneous graph A that captures the interactions between users and food items based on observed rating data. The graph contains two types of nodes: user nodes and food nodes. An edge is created between a user node and a food node only if the user has rated that particular food item. The edge weight encodes the rating, normalized by dividing the original rating (ranging from 1 to 5) by 5, resulting in a value between 0.2 and 1.0.

The edge weight is defined as:

$$A_{\text{user,food}} = \frac{\text{rating}(\text{user, food})}{5} \quad (7)$$

There are no connections between user-user pairs or food-food pairs, so the corresponding entries in the adjacency matrix are zero. This structure forms a bipartite graph where all edges represent meaningful user-food interactions, and their weights directly reflect the users' preferences.

Since user and food features originate from different sources, their original feature dimensions—denoted as d_1 for users and d_2 for foods—are typically not aligned. To enable unified processing in the GCN, we apply separate linear transformations to project both user and food features into a shared embedding space of dimension d .

Let $U \in \mathbb{R}^{N \times d_1}$ represent the user feature matrix and $F \in \mathbb{R}^{M \times d_2}$ represent the food feature matrix. The transformed features are computed as:

$$h_u = W_u U + b_u, \quad h_f = W_f F + b_f \quad (8)$$

where $W_u \in \mathbb{R}^{d \times d_1}$ and $W_f \in \mathbb{R}^{d \times d_2}$ are learnable weight matrices, and $b_u, b_f \in \mathbb{R}^d$ are bias terms.

After this transformation, the user and food embeddings are concatenated into a single input feature matrix:

$$X = \text{concat}(h_u, h_f) \in \mathbb{R}^{(N+M) \times d} \quad (9)$$

This unified representation serves as the input node feature matrix for the subsequent graph convolutional layers.

To learn expressive representations for both users and food items, we apply a two-layer Graph Convolutional Network (GCN) to the constructed heterogeneous graph. Each GCN layer is composed of three sequential components: graph convolution (GraphConv), layer normalization (LayerNorm), and LeakyReLU activation.

The forward pass of the two GCN layers can be formulated as:

$$X^{(i+1)} = \hat{A} X^i W_{i+1}, \quad X^{(i+1)} = \text{LeakyReLU}(\text{LayerNorm}(X^{(i+1)})) \quad (10)$$

where $\hat{A}$ denotes the normalized adjacency matrix with added self-loops, W_1 and W_2 are learnable weight matrices for each layer, and X is the input node feature matrix.

The final output $X^{(2)}$ contains the updated embeddings for all nodes, incorporating both the node features and the structural information from the graph. The embedding vector for each user-food

pair is then extracted and concatenated, then passed through a fully connected layer to predict the rating:

$$\text{Emb} = \text{concat}(X^{(2)}[u], X^{(2)}[f]) \quad (11)$$

$$R = \sigma(W_{\text{reg}} \cdot \text{Emb} + b_{\text{reg}}) \times \text{max_rating} \quad (12)$$

The final output rating lies within the range of 1 to 5.

4 Experiments

We evaluate the performance of our graph-based recommendation models against traditional collaborative filtering, which is our baseline, using RMSE (Root Mean Square Error) and MAE (Mean Absolute Error) as metrics. Below is the result:

Method name	RMSE	MAE
Collaborative filtering (baseline)	1.1905	0.9173
Two-stage architecture	1.1443	0.8750
User GAT	1.0466	0.7769
User-food heterogeneous GCN	1.1170	0.8606

Table 1: Test results

The results demonstrate that:

1. Collaborative filtering shows the poorest performance, confirming the limitations of traditional methods in complex scenarios.
2. The Graph Attention Network (GAT) achieves optimal performance on both metrics, proving the effectiveness of attention-based user relationship modeling for recommendation tasks.
3. The two-stage architecture also delivers competitive results, suggesting multi-stage optimization improves performance, though still slightly inferior to GAT in our experiments.

5 Discussion

5.1 GAT-MLP (Graph Attention Network + Multi-Layer Perceptron)

The GAT-MLP model combines the strengths of Graph Attention Networks (GAT) and Multi-Layer Perceptrons (MLP) to effectively model complex user-item interactions. One key advantage of this architecture is its adaptive attention mechanism, which dynamically learns the importance of neighboring nodes—i.e., the relevance of user-item relationships—without relying on fixed edge weights. This allows the model to flexibly focus on critical interactions that contribute most to rating prediction. Additionally, the use of multi-head attention enhances the model’s expressiveness by capturing user preferences from multiple perspectives, leading to richer representations.

On top of the learned node embeddings, a nonlinear MLP is employed to model complex rating patterns, enabling the prediction of user preferences that may not follow a simple linear trend. This design also facilitates the integration of additional user or item features—such as gender, food category, or other contextual attributes—which further improves the model’s understanding of preference behavior.

However, the GAT-MLP model also has disadvantages. Due to the attention mechanism and deep MLP layers, the model incurs a high computational cost and often requires GPU acceleration for

efficient training. Moreover, the model introduces a number of hyperparameters—including the number of attention heads, hidden dimensions, and MLP layer settings—making it more sensitive to tuning and potentially harder to optimize.

5.2 Hetero-GCN (Weighted Heterogeneous Graph Convolutional Network)

Hetero-GCN builds on the idea of modeling users and items as distinct node types in a heterogeneous graph. By applying separate linear transformations to project these features into a shared embedding space, the model preserves the semantic differences while enabling effective message passing. One key advantage of this approach is its robustness to the cold-start problem: since the model explicitly uses user and item features, it can still make reasonable predictions even when interaction data is sparse or missing.

In terms of efficiency, Hetero-GCN offers a solid trade-off. It is typically faster to train than attention-based models like GAT-MLP, while still delivering competitive accuracy. This makes it a practical choice when computational resources are limited or when rapid model iteration is required.

However, there are also limitations. The model relies on static edge weights—usually normalized rating values—which means it cannot dynamically adjust the strength of interactions during training. In contrast, attention mechanisms can learn to focus on more informative relationships. Additionally, like many GCN-based models, Hetero-GCN is susceptible to the over-smoothing problem: stacking too many layers can cause node representations to become indistinguishable, reducing the model’s ability to capture meaningful differences between users and items.

Overall, Hetero-GCN is a well-balanced model that combines efficiency with the ability to incorporate rich node features, but it may fall short in scenarios that require fine-grained modeling of interaction strength or deep relational patterns.

5.3 KNN-GAT (K-Nearest Neighbors Graph Attention Network)

KNN-GAT leverages the local structure of user-user similarity graphs, making it particularly interpretable. Since each node aggregates information from its K most similar neighbors, the model’s recommendation logic can often be understood in human terms—such as “users similar to you liked this item.” This interpretability, combined with the ability to function without dense feature information, makes KNN-GAT robust in scenarios where only interaction data is available.

However, its reliance on heuristic similarity measures introduces some limitations. The hard cutoff in selecting K neighbors may lead to the inclusion of noisy or irrelevant nodes, while entirely ignoring potentially informative global patterns beyond the neighborhood. Compared to GAT-MLP, which dynamically learns attention weights across all connected nodes, KNN-GAT’s fixed neighbor structure may limit its expressiveness, especially in more complex interaction settings.

6 Conclusion

This project investigates graph-based models for food recommendation, using real-world data from 418 users. We design three architectures: a user-only GAT, a two-stage KNN-GAT+MLP, and a heterogeneous GCN model. All models outperform traditional collaborative filtering, with the GAT-based approaches demonstrating superior adaptability and accuracy. Each model reflects a trade-off between interpretability, complexity, and generalization. Our findings highlight the potential of GNNs in personalized recommendation, and suggest future directions in improving efficiency and dynamic edge modeling.

7 Appendix

7.1 Questionnaire link

The questionnaire used in this project can be accessed at [link](#).

7.2 Algorithm for Collaborative Filtering (Baseline)

Algorithm 1 Collaborative filtering

Input: $R \in \mathbb{R}^{m \times n}$ (Score matrix where each row represents a user's scores on foods), k (Number of similar users considered in prediction)

Output: R^{pred} (Score matrix where originally 0 scores are predicted by the algorithm)

- 1: For each pair of users, measure their similarity using cosine similarity $S_{ij} = \frac{R_i^\top R_j}{\|R_i\| \|R_j\|}$, where $i, j \in \{1, \dots, n\}$ and $i \neq j$.
 - 2: $R^{\text{pred}} \leftarrow \mathbf{0}_{m \times n}$. ▷ Initialize the prediction matrix.
 - 3: **for** $i \in \{1, \dots, m\}$ **do**
 - 4: **for** $j \in \{1, \dots, n\}$ **do**
 - 5: **if** $R_{ij} > 0$ **then**
 - 6: $R_{ij}^{\text{pred}} \leftarrow R_{ij}$. ▷ The score already exists so does not need prediction.
 - 7: **else**
 - 8: For each user $i \in \{1, \dots, m\}$, find the most similar k users as $\mathcal{N}_i = \{u_1, \dots, u_k\}$ where $S_{iu_1} \geq S_{iu_2} \geq \dots \geq S_{iu_k}$.
 - 9: Among the the most similar k users of user i , select the similar users that has scores on food j , as $\hat{\mathcal{N}}_i = \{\hat{u}_1, \dots, \hat{u}_{k'}\}$ where k' denotes the size of $\hat{\mathcal{N}}_i$.
 - 10: Predict the score of user i on food j : $R_{ij}^{\text{pred}} \leftarrow \frac{1}{k'} \sum_{u=1}^{k'} S_{iu} R_{uj}$. ▷ Mean of weighted sum of scores by users in $\hat{\mathcal{N}}_i$.
 - 11: $R_{ij}^{\text{pred}} \leftarrow \text{round}(\text{clamp}(R_{ij}^{\text{pred}}, 1, 5))$. ▷ Ensure that the predicted score is an integer in $[1, 5]$.
 - 12: **return** R^{pred}
-

7.3 Algorithm for User GAT Method

Algorithm 2 GAT-based User Rating Prediction

Input: $X \in \mathbb{R}^{N \times d}$, edge_index, $R \in \mathbb{R}^{N \times M}$, train_mask, H , h

Output: Predicted rating matrix $\hat{R} \in \mathbb{R}^{N \times M}$

- 1: Initialize parameters: $\{W_h^{(1)}, a_h^{(1)}\}_{h=1}^H$, $W^{(2)}$, W_{out}
 - 2: **for** each node i **do**
 - 3: **for** each neighbor $j \in \mathcal{N}(i)$ **do**
 - 4: Compute attention coefficients by Eq. (1)
 - 5: Aggregate features by Eq. (2)
 - 6: Concatenate multi-head outputs: $z_i \leftarrow \parallel_{h=1}^H z_i^{(h)}$
 - 7: Second layer: $h_i \leftarrow \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(2)} W^{(2)} z_j$
 - 8: Predict ratings: $\hat{R}_i \leftarrow W_{\text{out}} h_i$
 - 9: Compute MSE loss by Eq. (4)
 - 10: Backpropagate and update parameters
-

7.4 Two-stage Architecture

7.4.1 Flowchart for the Architecture

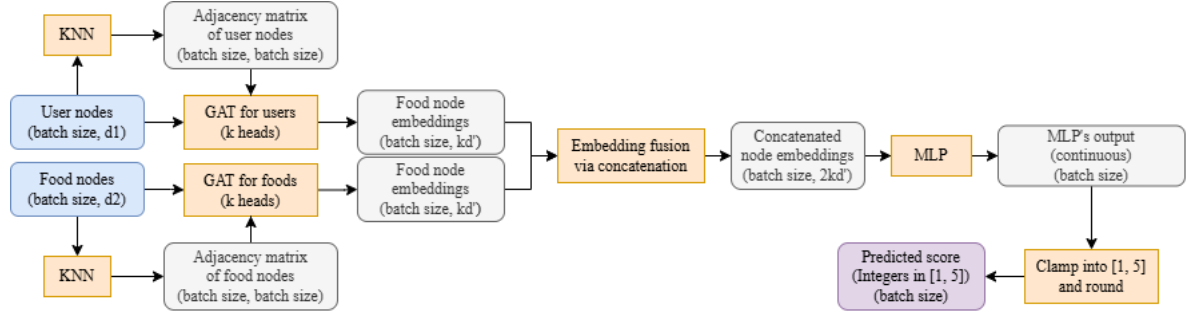


Figure 1: The two stage architecture of food recommender

7.4.2 Algorithms for the Architecture's Components

Algorithm 3 Compute adjacency matrix through KNN

Input: $X \in \mathbb{R}^{n \times d}$ (Node feature matrix), k (Number of neighbors)

Output: A^{norm} (Normalized adjacency matrix)

- 1: For each pair of nodes, calculate their Euclidean distance $S_{ij} = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$, where $i, j \in \{1, \dots, n\}$ and $i \neq j$.
 - 2: For each node $i \in \{1, \dots, n\}$, choose $(k + 1)$ nearest neighbors as $\mathcal{N}_i = \{j_1, j_2, \dots, j_{k+1}\}$ where $S_{ij_1} \geq S_{ij_2} \geq \dots \geq S_{ij_{k+1}}$.
 - 3: Initialize the adjacency matrix as $A^{\text{raw}} \in \{0, 1\}^{n \times n}$ where $A_{ij}^{\text{raw}} = 1$ if $j \in \mathcal{N}_i$ and $i \neq j$, otherwise $A_{ij}^{\text{raw}} = 0$.
 - 4: Symmetrize the adjacency matrix by taking the element-wise maximum, i.e. $A_{ij}^{\text{sym}} = \max(A_{ij}^{\text{raw}}, A_{ji}^{\text{raw}})$. ▷ To ensure the undirected graph property.
 - 5: Compute the degree matrix D where $D_{ii} = \sum_{j=1}^n A_{ij}^{\text{sym}}$ and $D_{ij} = 0$ where $i, j \in \{1, \dots, n\}$ and $i \neq j$.
 - 6: Normalize the adjacency matrix: $A^{\text{norm}} = D^{-1/2}(A^{\text{sym}} + I)D^{-1/2}$.
 - 7: **return** A^{norm}
-

Algorithm 4 Graph Attention Network (GAT) Forward Propagation

Input: $X \in \mathbb{R}^{N \times d}$ (Node feature matrix); $A \in \{0, 1\}^{N \times N}$ (Adjacency matrix); K (Number of attention heads); d' (Output dimension per head); α (LeakyReLU negative slope)

Output: $Z \in \mathbb{R}^{N \times Kd'}$ (Node embeddings)

```
1: Initialize  $Z \leftarrow \text{EmptyMatrix}(N, Kd')$ 
2: for  $k = 1$  to  $K$  do
3:    $W^k \leftarrow \text{LearnableMatrix}(d \times d')$ 
4:    $h^k \leftarrow XW^k$  ▷ Linear projection
5:    $a^k \leftarrow \text{LearnableVector}(2d')$  ▷ Compute attention coefficients
6:    $E^k \leftarrow \text{ZeroMatrix}(N \times N)$ 
7:   for  $i = 1$  to  $N$  do
8:     for  $j = 1$  to  $N$  do
9:       if  $A_{ij} = 1$  or  $i = j$  then
10:         $e_{ij}^k \leftarrow \text{LeakyReLU}(a^k[h_i^k \| h_j^k])$  ▷ Calculate the neighbor pair's attention coefficient,
        where “ $\|$ ” means concatenation
11:         $E_{ij}^k \leftarrow e_{ij}^k$ 
12:      else
13:         $E_{ij}^k \leftarrow -\infty$  ▷ This pair of nodes are not neighbors so they do not have an attention
        coefficient
14:    $\alpha^k \leftarrow \text{softmax}(E^k, \text{dim} = 1)$  ▷ Normalize attention weights using row-wise softmax
15:    $z^k \leftarrow \alpha^k h^k$  ▷ Aggregate neighbors
16:    $Z[:, (k-1)d' : kd'] \leftarrow z^k$  ▷ Concatenate heads as the final embedding
17: return  $Z$ 
```

Algorithm 5 Forward propagation through MLP

Input: $Z \in \mathbb{R}^{n \times d_{\text{emb}}}$ (Node embedding matrix), h (Number of hidden layers) $L \in \mathbb{R}^h$ (Hidden layers' dimensions)

Output: $\hat{r}_{\text{valid}} \in \mathbb{R}^n$ (Predicted scores)

```
1:  $H^{(0)} = Z$ .
2: for  $l \in \{1, \dots, h\}$  do
3:   Propagation on the current hidden layer:  $H^{(l)} = \text{ReLU}(W^{(l)}H^{(l-1)} + b^{(l)})$ , where  $W^{(l)} \in \mathbb{R}^{L_l \times L_{(l-1)}}$  and  $b^{(l)} \in \mathbb{R}^{L_l}$ .
4: Convolution on the output layer:  $\hat{r} = W^{(h+1)}H^{(h)} + b^{(h+1)}$ , where  $W^{(h+1)} \in \mathbb{R}^{L_h \times n}$  and  $b^{(h+1)} \in \mathbb{R}^n$ .
5: Transform all predicted scores to integers in  $[1, 5]$ :  $\hat{r}_{\text{valid}} = \text{round}(\text{clamp}(\hat{r}, 1, 5))$ .
6: return  $\hat{r}_{\text{valid}}$ 
```

7.5 Algorithm for User-food Heterogeneous GCN

Algorithm 6 User-food heterogeneous GCN with weight

Input: User features U , Food features F , Edge index E , Edge weights W

Output: Predicted ratings R

- 1: Initialize projections:
 - 2: $h_u = W_u U + b_u, h_f = W_f F + b_f$
 - 3: Concatenate user and food features: $X = \text{concat}(h_u, h_f)$
 - 4: Apply GCN with edge weights:
 - 5: $X^{(1)} = \hat{A} X W_1, X^{(1)} = \text{LeakyReLU}(\text{LayerNorm}(X^{(1)}))$
 - 6: $X^{(2)} = \hat{A} X^{(1)} W_2, X^{(2)} = \text{LeakyReLU}(\text{LayerNorm}(X^{(2)}))$
 - 7: Compute embeddings: $Emb = \text{concat}(X^{(2)}[u], X^{(2)}[f])$
 - 8: Predict rating: $R = \sigma(W_{\text{reg}} Emb + b_{\text{reg}}) \times \text{max_rating}$
 - 9: **return** R
-

References

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2017). Graph attention networks. *arXiv preprint arXiv:1710.10903*, abs/1710.10903. Version 3.